

SOFTWARE REQUIREMENTS ANALYSIS (SRA)

Invoice Management Module - Iconic Hishab Accounting Software

Prepared date: 14-Sep-2026

Version: 1.0

Analysis basis: implemented invoice module source, current UI feedback, and accounting workflow requirements.

1. Executive Summary

=====

The invoice module is a branch-aware accounting workflow that turns customer billing activity into controlled accounting vouchers. The main business challenge is preserving audit history while allowing practical edits to invoice items and payments. Invoice item edits should be recalculation events; payment/refund events should remain independent history entries linked to vouchers.

2. Stakeholder Needs

=====

Stakeholder | Need | Module Response

Business owner | Accurate customer invoice and due tracking | List, detail, due status, payment history, and summary report.

Accounts user | Collect partial payments later without losing previous records | Date-wise payment history and receipt vouchers.

Manager/Auditor | Trace every financial change | Voucher links and invoice audit logs.

Admin | Control role access | Invoice module permissions: create/read/update/delete.

Mobile/laptop user | Usable list screen on small devices | Responsive table/card layout with horizontal overflow protection.

3. Workflow Analysis

=====

- Create Invoice: user selects customer from Particular Accounts, enters item details, optional initial paid amount, then creates final invoice or saves draft.

- Draft Flow: draft can be edited and later finalized; vouchers are created only during finalization.

- Final Invoice Edit: item rate/price and quantity can be edited; existing payment history stays intact; totals and status are recalculated.

- Payment Flow: due collection, refund, and payment edit are operational actions on Edit Invoice page; View Invoice shows history and related vouchers.

- Report Flow: Invoice Summary Report aggregates invoice totals, paid, refund, due, and status rows by filters.

4. Requirement Prioritization

=====

Requirement Area | Priority | Reason

Accurate voucher posting | Must Have | Core accounting impact; wrong vouchers affect ledgers.

Payment/refund history | Must Have | Partial payment business flow requires date-wise traceability.

Draft/finalize separation | Must Have | Prevents premature accounting entries.

Responsive invoice list | Should Have | Operational usability across devices.

Print/PDF | Should Have | Required for customer-facing invoice/report output.

Customer create shortcut | Could Have | Improves user efficiency but does not change accounting logic.

5. Data and Calculation Analysis

=====

The module uses Invoice as the master record, InvoiceItem for line-level billing data, InvoicePayment for payment/refund history, InvoiceVoucher for accounting posting links, and InvoiceAuditLog for traceability. Paid amount should be calculated from non-refund payments minus refunds or stored paidAmount synchronized after each event. Due amount equals grandTotal minus paidAmount and should not rely on UI-only values.

Scenario | Expected Calculation

Grand total 10,000; payments 3,000 + 2,000 | Paid 5,000; Due 5,000; Status PARTIALLY_PAID. Then item edit makes grand total 5,000 | Paid 5,000; Due 0; Status PAID; payment history remains.

Refund 1,000 after paid 5,000 | Paid/net collected 4,000; refund total 1,000; Due recalculated according to grand total.

Draft saved with initial paid amount | No voucher until finalization; finalization posts initial paid/due vouchers.

6. Risk Analysis

Risk | Impact | Mitigation

Invoice total edited below collected amount | Incorrect paid/due state or overpayment | Block the update or require refund/adjustment before allowing it.

Deleting finalized invoice | Audit/report data loss | Restrict delete to Draft and Cancelled statuses.

Voucher reversal not linked | Ledger mismatch | Create InvoiceVoucher records with postingType REVERSAL and audit log entries.

Role missing Invoice permission after deployment | Menu/actions hidden for users | Seed/sync Invoice permissions and assign role permissions.

Small-screen table clipping | User cannot access actions | Use responsive card view or horizontal scroll with min-width table.

Production schema not migrated | 500 errors on invoice APIs | Run migration/schema SQL before enabling module in production.

7. API and Security Analysis

=====

- All invoice APIs require JWT verification.
- Create/update/payment/refund/cancel/delete operations require selected branch and branch permission validation.
- Role permissions map to Module Invoice with create, read, update, and delete actions.
- The frontend injects selectedBranchId into invoice API requests from local storage.

8. UI Placement Analysis

=====

Page | Should Contain | Should Not Contain

Invoice List | Search/filter, customer column, item names column, status, view/edit/cancel/delete/restore as allowed | Print/PDF buttons.

View Invoice | Top buttons, read-only items, payment history, related vouchers | Collect due payment form, refund form, payment history edit action.

Edit Invoice | Editable item price/quantity, collect due payment, refund form, payment history edit/action | Duplicate read-only report-only blocks that confuse edit workflow.

Summary Report | Filters, totals, refund total, list rows, print/PDF | Direct mutation actions.

9. Open Items and Recommendations

=====

- Confirm whether overpayment after invoice total reduction should be blocked or automatically create credit/refund workflow.
- Add automated tests for payment edit, refund report total, draft finalization, and cancel reversal.
- Keep production deployment checklist separate from feature development because Docker changes should not be touched without approval.